

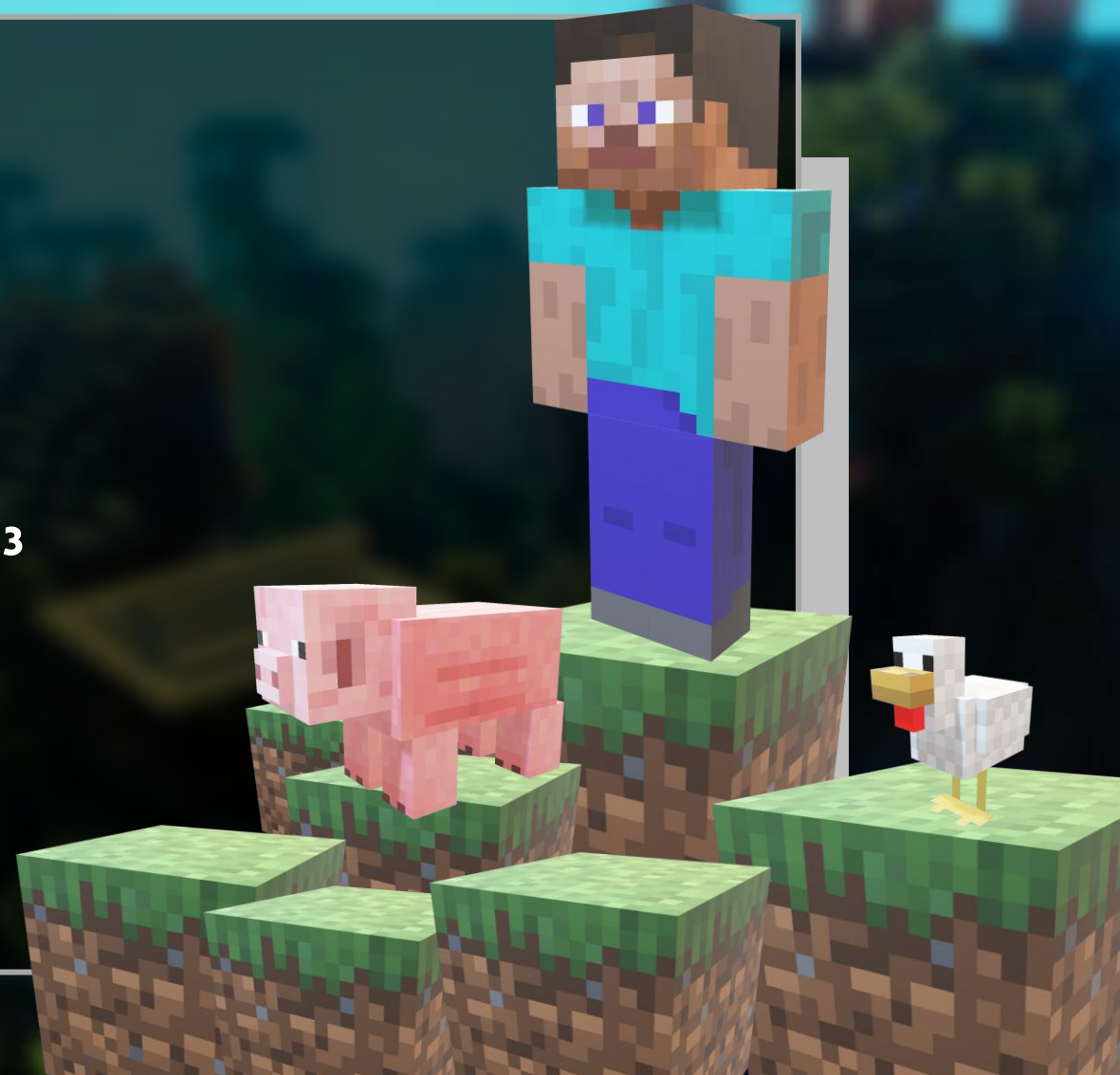
FIACRAFT

Progetto FIA 2025/2026

Loading...

CHI SIAMO

Giovanni Lavorato 0512116371
Marco Battaglia 0512116374
Domenico di Caprio 0512118543



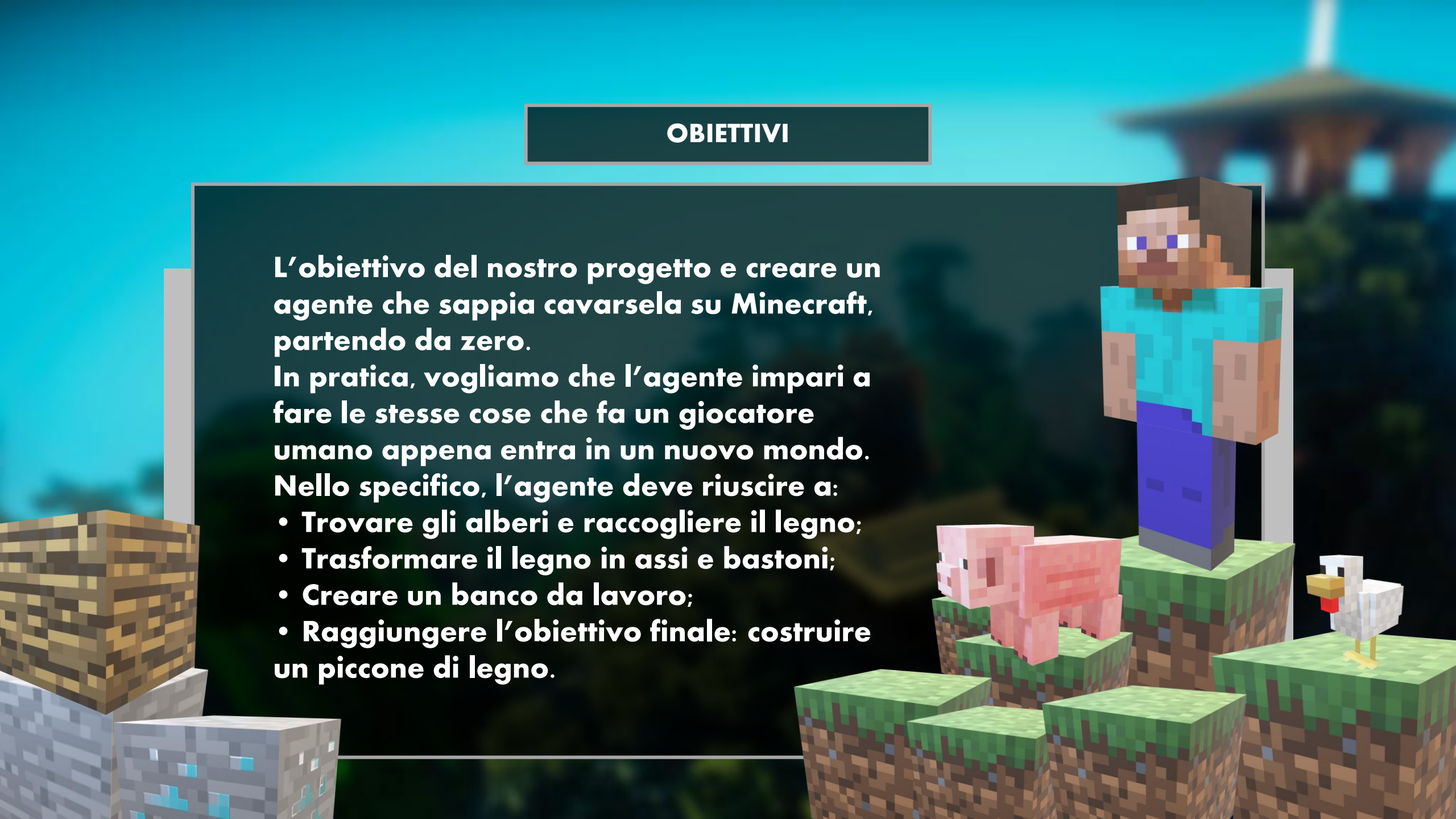
OBIETTIVI

L'obiettivo del nostro progetto è creare un agente che sappia cavarsela su Minecraft, partendo da zero.

In pratica, vogliamo che l'agente impari a fare le stesse cose che fa un giocatore umano appena entra in un nuovo mondo.

Nello specifico, l'agente deve riuscire a:

- **Trovare gli alberi e raccogliere il legno;**
- **Trasformare il legno in assi e bastoni;**
- **Creare un banco da lavoro;**
- **Raggiungere l'obiettivo finale: costruire un piccone di legno.**



SPECIFICA P.E.A.S

Performance

Enviroment

Actuators

Sensors



Performance

- 1 - **Creazione autonoma del piccone di legno**
 - Esecuzione dei task per raggiungere l'obiettivo
- 2 - **Riduzione di azioni inutili o casuali**
 - Efficienza nel raggiungimento dell'obiettivo



Enviroment

- **Mondo 3D dinamico di MineRL**
- **Parzialmente osservabile (prima persona)**
- **Stocastico e sequenziale (ambiente randomizzato sequenza di crafting)**
- **Singolo agente**
- **Misto (gestione dei vari tasti e movimenti fluidi per il mouse)**



Attuatori

Mezzi con cui l'agente agisce nel mondo.

Azioni disponibili:

- **Movimento:** forward, back, left, right
- **Rotazione visuale** (camera)
- **JumpBreak block** (raccolta legno)
- **Place block** (crafting table)
- **Open inventory**
- **Craft** (planks, stick, crafting table, wooden pickaxe)



Sensori

**L'agente riceve informazioni attraverso
la visuale in prima persona e i dati
nell'inventario**

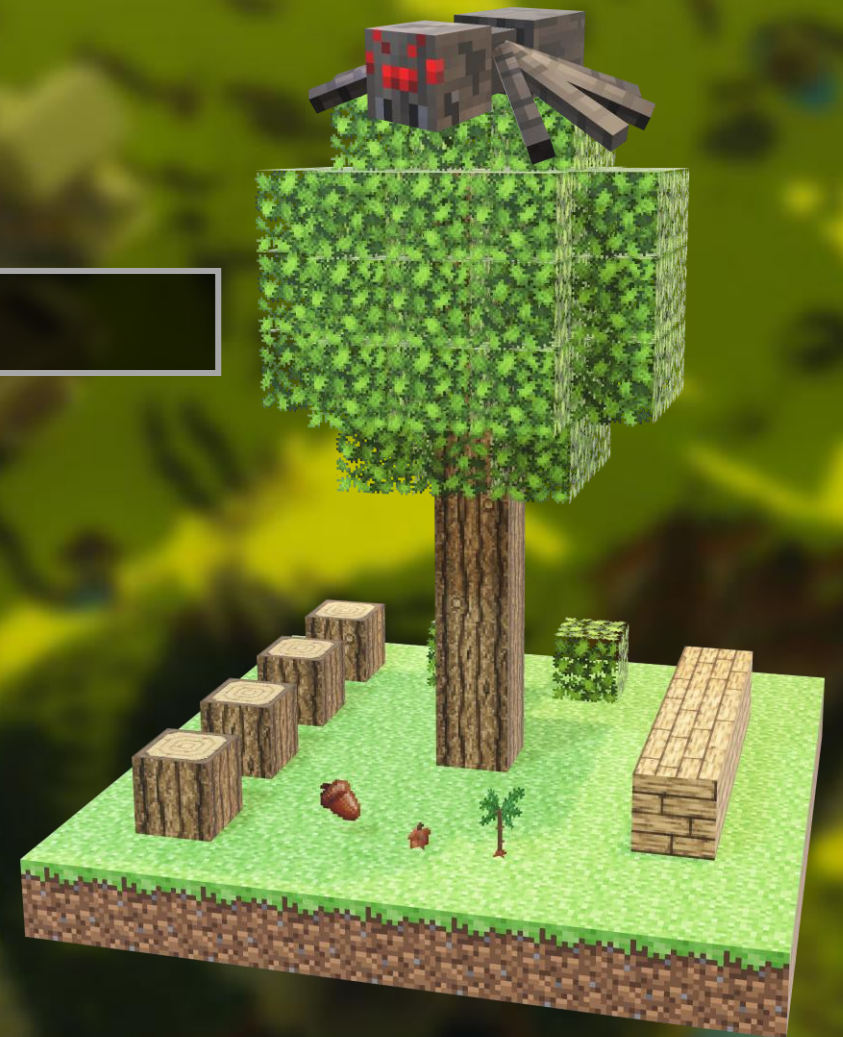


Sfide tecniche

- 1) Grande mole di dati da filtrare
- 2) Tempo necessario e gerarchia delle azioni per la creazione di un piccone in legno
- 3) Minecraft essendo un mondo dinamico rischia di far confondere l'agente



SOLUZIONE SFIDA 1



Dati

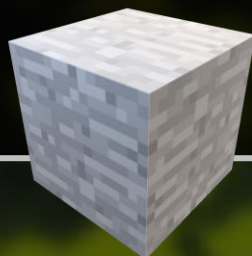
Download Dataset

Download dei video messi a disposizione da un dataset pre-esistente



Discard

Scartare video di sessioni già avviate lasciando solo quelle con l'inventario vuoto



Cut Data

Tagliare i video al raggiungimento del nostro obiettivo



SOLUZIONE SFIDA 2



Behavioural Cloning

Il **Behavioural Cloning** è una tecnica di **imitation learning** in cui un agente impara osservando delle **dimostrazioni** invece di esplorare l'ambiente da solo.



Behavioural Cloning

```
pi_distribution, _, new_agent_state = policy.get_output_for_observation(
    agent_obs, agent_state, first_tensor
)

with th.no_grad():
    original_pi_distribution, _, _ = original_policy.get_output_for_observation(
        agent_obs, agent_state, first_tensor
    )

log_prob = policy.get_logprob_of_action(pi_distribution, agent_action)
kl_div = policy.get_kl_of_action_dists(pi_distribution, original_pi_distribution)

current_nll = -log_prob
step_loss = current_nll + KL_LOSS_WEIGHT * kl_div

seq_loss += step_loss
seq_nll += current_nll
seq_kl += kl_div

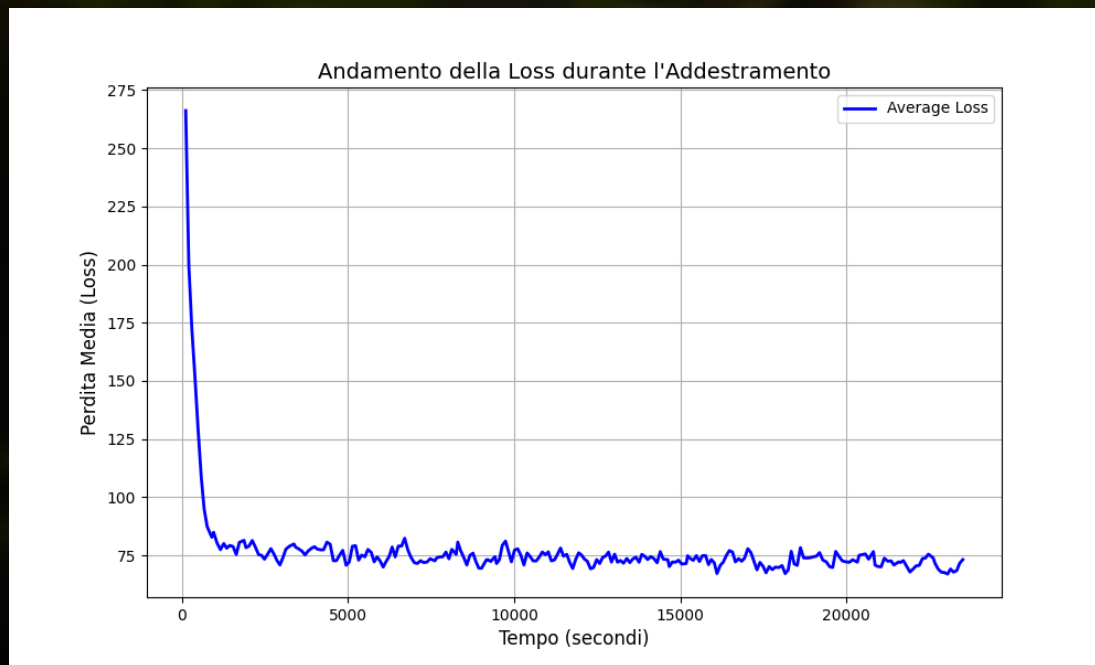
agent_state = new_agent_state

if valid_steps > 0:
    final_loss = (seq_loss / valid_steps) / BATCH_SIZE
    final_loss.backward()
```

L'agente predice una distribuzione di probabilità sulle azioni, calcoliamo la Negative Log Likelihood rispetto all'azione umana e aggiungiamo una KL Divergence per mantenere stabilità rispetto al modello iniziale. Infine facciamo backpropagation per aggiornare i pesi.



BC Loss graph



L'andamento della Loss durante le fasi del **Behavioural Cloning**



Reinforcement Learning

Il **Reinforcement Learning** è una tecnica di machine learning in cui un agente impara **interagendo con l'ambiente** e ricevendo **reward** in base alle azioni che esegue.



Reinforcement Learning

```
for step in range(max_steps):
    action = agent.get_action(obs)
    if "ESC" not in action: action["ESC"] = 0

    next_obs, _, done, _ = env.step(action)

    inv_obs = next_obs.get("inventory", None)
    reward_step, task_completed = compute_reward_and_stop(inv_obs, best_inventory, (step==0))
    cumulative_reward += reward_step

    print(f"\rStep: {step:4d} | Rew: {reward_step:.2f} | Tot: {cumulative_reward:.2f}", end="", flush=True)

    # Training su GPU
    agent_obs = tree_map(lambda x: x.to(device) if isinstance(x, th.Tensor) else x, agent._env_obs_to_agent(obs))
    agent_action = tree_map(lambda x: x.to(device) if isinstance(x, th.Tensor) else x, agent._env_action_to_agent(action, to_torch=True))

    pi_logits, _, _ = agent.policy.get_output_for_observation(agent_obs, agent.hidden_state, th.tensor([False], device=device))
    log_prob = agent.policy.get_logprob_of_action(pi_logits, agent_action)

    batch_log_probs.append(log_prob)
    batch_advantages.append(th.tensor([reward_step], device=device, dtype=th.float32))

    if (step + 1) % 64 == 0 or done or task_completed:
        if len(batch_log_probs) > 0:
            # Spostiamo tutto su GPU esplicitamente per il calcolo perdita
            adv_t = normalize(th.stack(batch_advantages).to(device))
            probs_t = th.stack(batch_log_probs).to(device)

            loss = -(probs_t * adv_t.detach()).mean()

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
```

Durante il Reinforcement Learning l'agente interagisce direttamente con l'ambiente Minecraft. Dopo ogni azione riceve un reward basato sull'inventario. Calcoliamo la probabilità logaritmica dell'azione eseguita e aggiorniamo la policy usando policy gradient per massimizzare il reward cumulativo.



SOLUZIONE SFIDA 3



Data Augmentation — Mirroring

Il **mirroring** è una tecnica di data augmentation che consiste nel **specchiare orizzontalmente le immagini** del dataset.



Data Augmentation — Mirroring

```
if "mouse" in data:
    data["mouse"]["dx"] = -data["mouse"]["dx"]
    data["mouse"]["scaledx"] = -data["mouse"]["scaledx"]
    data["mouse"]["x"] = width - data["mouse"]["x"]

if "keyboard" in data and "keys" in data["keyboard"]:
    keys = data["keyboard"]["keys"]
    new_keys = []
    new_chars = []
    for key in keys:
        if key == "key.keyboard.a":
            new_keys.append("key.keyboard.d")
            new_chars.append("d")
        elif key == "key.keyboard.d":
            new_keys.append("key.keyboard.a")
            new_chars.append("a")
        else:
            new_keys.append(key)
            if "chars" in data["keyboard"]:
                new_chars.append(data["keyboard"]["chars"])

    data["keyboard"]["keys"] = new_keys
    if "chars" in data["keyboard"]:
        data["keyboard"]["chars"] = "".join(new_chars)

if "hotbar" in data:
    data["hotbar"] = 8 - data["hotbar"]
```

Per aumentare il dataset abbiamo specchiato i video orizzontalmente. Oltre alle immagini, abbiamo specchiato anche le azioni del giocatore, invertendo il movimento del mouse e scambiando i tasti A e D, in modo da mantenere la coerenza tra osservazioni e azioni.



Testing

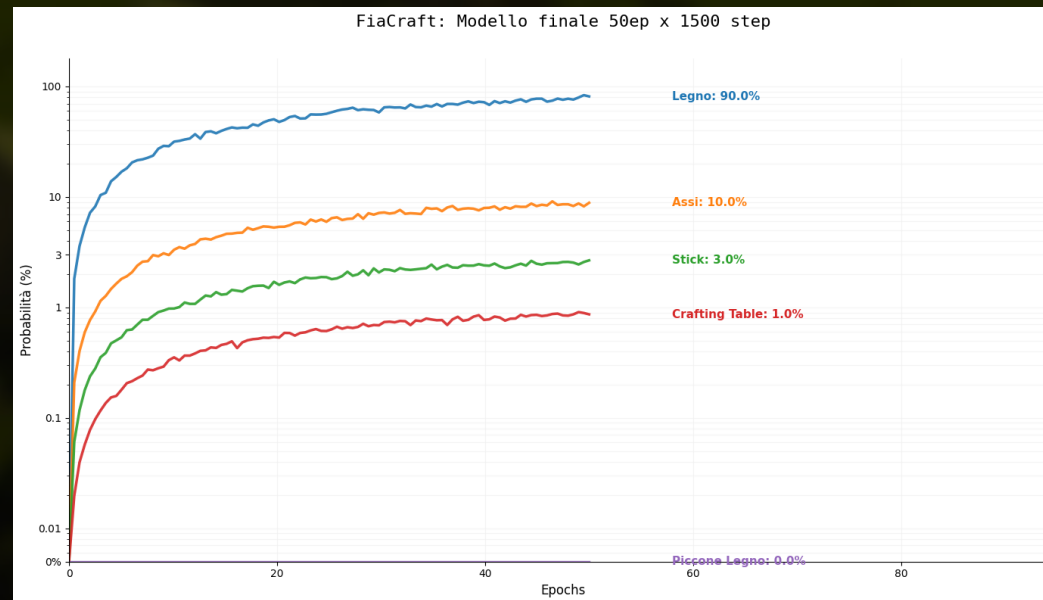


Testing

Di seguito sono riportati i risultati ottenuti dai test condotti su 50 episodi, ciascuno della durata di 1500 step.

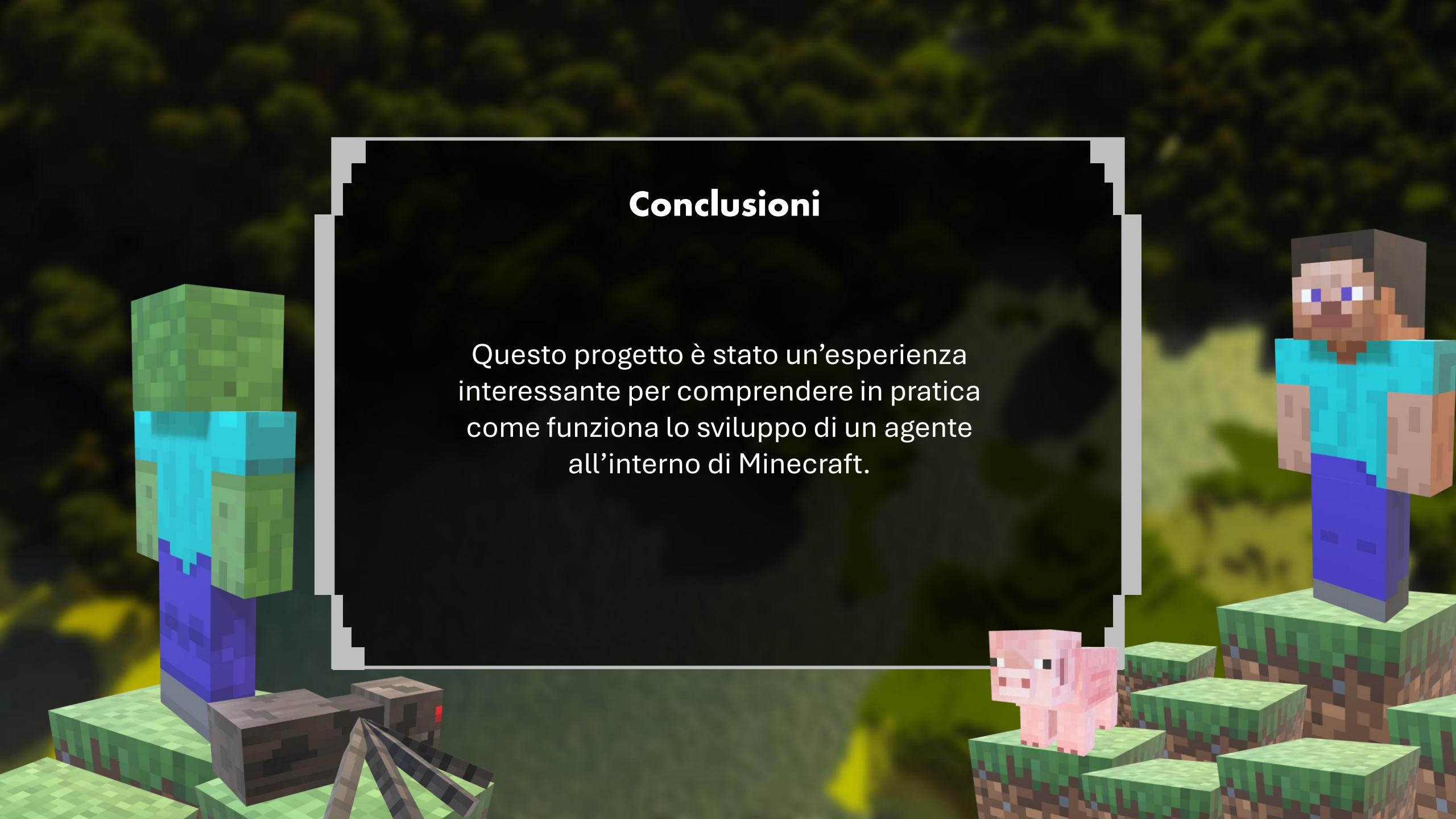


Testing



Conclusioni

Questo progetto è stato un'esperienza interessante per comprendere in pratica come funziona lo sviluppo di un agente all'interno di Minecraft.



thank you

